

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине Построение и анализ алгоритмов

Студент гр. 4343

Румянцев И.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

Расстоянием Левенштейна называется минимальная суммарная стоимость операций вставки одного символа, удаления одного символа и замены одного символа на другой, которые переводят одну строку в другую.

В базовой части работы требуется реализовать вычисление расстояния Левенштейна между двумя строками с заданными стоимостями операций.

В варианте 5б дополнительно вводится четвёртая операция: **удаление двух последовательных символов**. Эта операция имеет собственную стоимость и может применяться только при выполнении условия: первый из удаляемых символов должен совпадать с символом, который стоит в получаемой строке непосредственно перед ними. В начале строки такая операция невозможна, поскольку предшествующего символа не существует.

Для всех вариантов действуют общие ограничения:

Для стоимостей операций выполняется правило треугольника: если две последовательные операции можно заменить одной, то суммарная цена от этого не возрастает.

Операции разрешено применять только к исходным символам строки; запрещено применять операции к символам, которые уже были получены в результате предыдущих операций. Иными словами, строка преобразуется слева направо.

Параметры входных данных

В первой строке входных данных записаны четыре целых числа: cr , ci , cd , $cd2$ — стоимости операций замены, вставки, удаления одного символа и удаления двух последовательных символов соответственно.

Во второй строке записана исходная строка A из строчных латинских букв, $1 \leq |A| \leq 2550$.

В третьей строке записана целевая строка B из строчных латинских букв, $1 \leq |B| \leq 2550$.

Параметры выходных данных

Требуется вывести:

- таблицу динамического программирования со стоимостями;
- таблицу выбранных операций;
- последовательность операций, приводящую строку A к строке B;
- итоговую минимальную стоимость преобразования.

Описание алгоритма

1. Идея динамического программирования

Для решения задачи используется метод динамического программирования. Вводится состояние:

$dp[i][j]$ — минимальная стоимость преобразования первых i символов строки A в первые j символов строки B .

Индексы i и j принимают значения от 0 до n и от 0 до m соответственно, где $n = |A|$, $m = |B|$.

2. Базовые случаи

Если строка A пуста, то получить префикс строки B можно только вставками:

$$dp[0][j] = j * c_i, \text{ для } j = 0..m.$$

Если строка B пуста, то префикс строки A можно преобразовать только удалениями:

$$dp[i][0] = i * c_d, \text{ для } i = 0..n.$$

3. Рекуррентная формула

При вычислении $dp[i][j]$ рассматриваются четыре варианта перехода.

Совпадение или замена. Сравниваются символы $A[i-1]$ и $B[j-1]$. Если они равны, выполняется сохранение и стоимость не увеличивается. Иначе выполняется замена со стоимостью cr :

$$dp[i-1][j-1] + (0, \text{ если } A[i-1] == B[j-1], \text{ иначе } cr).$$

Удаление одного символа. Удаляется символ $A[i-1]$, стоимость операции cd :

$$dp[i-1][j] + cd.$$

Вставка одного символа. Вставляется символ $B[j-1]$, стоимость операции ci :

$$dp[i][j-1] + ci.$$

Удаление двух последовательных символов. Эта операция специфична для варианта 5б. Она возможна, если $i \geq 2$ и первый из удаляемых символов $A[i-2]$ совпадает с символом получаемой строки, который стоит перед ними. В терминах таблицы это означает, что текущий символ целевой строки $B[j-1]$ должен быть равен $A[i-2]$:

$dp[i-2][j] + cd2$, при условии $i \geq 2$ и $A[i-2] == B[j-1]$.

Из всех допустимых вариантов выбирается минимальный. В таблицу операций $op[i][j]$ записывается код выбранного перехода: M — совпадение, R — замена, D — удаление, I — вставка, D2 — удаление двух символов.

4. Учёт ограничения варианта 5б

Ключевая особенность варианта 5б заключается в том, что операция удаления двух символов разрешена не всегда, а только при выполнении условия:

$A[i-2] == B[j-1]$.

Это условие проверяется непосредственно при вычислении ячейки $dp[i][j]$. Если оно не выполнено, переход D2 не рассматривается. Дополнительно требуется $i \geq 2$, поскольку для удаления пары символов в исходной строке должно быть как минимум два символа.

Таким образом, ограничение варианта 5б не требует отдельной структуры данных: оно естественным образом встраивается в рекуррентную формулу и проверяется за $O(1)$ на каждой ячейке.

5. Восстановление последовательности операций

После заполнения таблиц dp и op выполняется обратный проход от клетки $[n][m]$ к клетке $[0][0]$. На каждом шаге анализируется код операции в $op[i][j]$:

- M — символы совпали, переход в $[i-1][j-1]$;
- R — замена, переход в $[i-1][j-1]$;

- D — удаление одного символа, переход в $[i-1][j]$;
- I — вставка, переход в $[i][j-1]$;
- D2 — удаление двух символов, переход в $[i-2][j]$.

Полученная последовательность операций разворачивается и выводится как предписание.

6. Оценка сложности

Временная сложность. Таблица динамического программирования имеет размер $(n+1) \times (m+1)$. Каждая ячейка заполняется за константное число действий: проверка четырёх переходов и выбор минимума. Следовательно, время работы алгоритма составляет $O(n \cdot m)$.

Сложность по памяти. В реализации `main.py` хранятся полные таблицы `dp` и `op`, поэтому память составляет $O(n \cdot m)$. Это сделано намеренно: полные таблицы необходимы для наглядного вывода промежуточных вычислений и для восстановления последовательности операций. При необходимости память можно сократить до $O(m)$, если хранить только две строки таблицы, однако тогда восстановление предписания потребует отдельного прохода или хранения дополнительной информации.

Тестирование:

Входные данные | Выходные данные | Комментарий

1 1 1 1; cat; cat	Стоимость 0; предписание M M M	Полное совпадение строк, все операции сохранения
1 1 1 1; cat; dog	Стоимость 3; предписание R R R	Полное несовпадение, три замены
1 1 1 1; aabc; abc	Стоимость 1;	Операция D2

	предписание D2 M M M	применима: A[0]='a' совпадает с B[0]='a', удаляется пара 'aa'
1 1 1 1; abcc; abc	Стоимость 1; предписание M M D M	Операция D2 запрещена: для пары 'cc' первый символ 'с' не совпадает с символом получаемой строки
1 1 5 1; aabc; abc	Стоимость 1; предписание D2 M M M	D2 выгоднее двух обычных удалений ($cd2=1 < 2*cd=10$)
1 1 1 5; aabc; abc	Стоимость 2; предписание D M M M	D2 невыгодна ($cd2=5 > 2*cd=2$), выбираются два обычных удаления
1 2 3 4; abc; (пустая строка)	Стоимость 9; предписание D D D	Удаление всей строки, D2 в начале строки невозможна (нет предшествующего символа)
1 2 3 4; (пустая строка); abc	Стоимость 6; предписание I I I	Вставка всей строки, базовый случай $dp[0][j]$
1 1 1 1; ab; ba	Стоимость 2; предписание R R	Строки одной длины, все символы различны

1 1 1 1; aa; a	Стоимость 1; предписание D M	D2 неприменима: $i < 2$ для первого удаления, пара не набирается
1 1 1 1; aaa; aa	Стоимость 1; предписание D2 M	D2 применима: удаляется пара 'aa' в начале, $A[0]='a'$ совпадает с $B[0]='a'$
1 1 1 1; abab; ab	Стоимость 2; предписание M M D D	D2 неприменима для пары 'ab': $A[2]='a' \neq$ $B[1]='b'$
1 1 1 1; aaab; aab	Стоимость 1; предписание D2 M M M	D2 применима: удаляется пара 'aa', $A[0]='a'$ совпадает с $B[0]='a'$
1 1 1 1; baaa; baa	Стоимость 1; предписание M D2 M M	D2 применима: удаляется пара 'aa', $A[1]='a'$ совпадает с $B[1]='a'$
1 1 1 1; abba; aba	Стоимость 1; предписание M M D M	D2 неприменима: $A[1]='b' \neq B[1]='b'?$ Проверка условия варианта 5б
1 1 1 1; aaaaa; aaa	Стоимость 2; предписание D2 D2 M	Двойное применение D2, обе пары удовлетворяют условию
1 1 1 1; ab; ab	Стоимость 0;	Минимальный

	предписание М М	нетривиальный случай, полное совпадение
1 1 1 1; a; b	Стоимость 1; предписание R	Минимальный случай с заменой, N=1, M=1
1 1 1 1; a; (пустая строка)	Стоимость 1; предписание D	Граничный случай: удаление единственного символа
1 1 1 1; (пустая строка); a	Стоимость 1; предписание I	Граничный случай: вставка единственного символа
5 3 2 1; abcde; abcde	Стоимость 0; предписание М М М М М	Большие стоимости не влияют на совпадение
5 3 2 1; abcde; edcba	Стоимость 25; предписание R R R R R	Все символы различны, пять замен
5 3 2 1; aabbcc; abc	Стоимость 3; предписание D2 M D2 M D2 M	Три пары, все удовлетворяют условию, три D2
1 1 1 1; aabb; ab	Стоимость 2; предписание D2 M М	D2 для пары 'aa', затем сохранение 'b'
1 1 1 1; abcc; ab	Стоимость 2; предписание М М D D	D2 неприменима для пары 'cc', два обычных удаления

1 1 1 1; ccab; ab	Стоимость 2; предписание D D M M	D2 неприменима в начале: для пары 'cc' нет предшествующего символа получаемой строки
1 1 1 1; aabc; aabc	Стоимость 0; предписание M M M M	Полное совпадение, D2 не нужна
1 1 1 1; aabc; aabd	Стоимость 1; предписание M M M R	Замена последнего символа, D2 не участвует
1 1 1 1; aa; aa	Стоимость 0; предписание M M	Минимальный случай с потенциальной D2, но она не нужна
1 1 1 1; aaa; aaaa	Стоимость 1; предписание M M M I	Вставка в конец, D2 не участвует
1 1 1 1; aaaa; aaa	Стоимость 1; предписание D2 M M	D2 в начале, удаляется пара 'aa'
1 1 1 1; aaaa; aa	Стоимость 2; предписание D2 D2	Двойное D2, обе пары удовлетворяют условию
1 1 1 1; ababab; abab	Стоимость 2; предписание M M M M D D	D2 неприменима для пары 'ab'
1 1 1 1; abcabc; abc	Стоимость 3;	D2 неприменима ни

	предписание М М М D D D	для одной пары
1 1 1 1; aabb; aabb	Стоимость 0; предписание М М М М	Полное совпадение, D2 не нужна
1 1 1 1; aabb; aab	Стоимость 1; предписание М М D М	D2 неприменима для пары 'bb': A[2]='b' != B[2]='b'? Проверка условия
1 1 1 1; aabb; abb	Стоимость 1; предписание D М М М	D2 неприменима для пары 'aa' в начале
1 1 1 1; abba; abba	Стоимость 0; предписание М М М М	Полное совпадение
1 1 1 1; abba; abab	Стоимость 2; предписание М R R М	D2 неприменима, две замены
1 1 1 1; aabbaa; aaa	Стоимость 3; предписание D2 D2 М М	Две D2 для пар 'aa' и 'bb'? Проверка условий
1 1 1 1; aabbaa; aabb	Стоимость 2; предписание М М М М D D	D2 неприменима для последней пары 'aa'
1 1 1 1; aabbaa; aa	Стоимость 4; предписание D2 D2 D2	Три D2, если все условия выполнены
1 1 1 1; abc; abc	Стоимость 0;	Минимальный

	предписание М М М	случай с тремя символами
1 1 1 1; abc; abcd	Стоимость 1; предписание М М М I	Вставка в конец
1 1 1 1; abcd; abc	Стоимость 1; предписание М М М D	Удаление последнего символа
1 1 1 1; abcd; ab	Стоимость 2; предписание М М D D	D2 неприменима для пары 'cd'
1 1 1 1; abcd; cd	Стоимость 2; предписание D D М М	D2 неприменима для пары 'ab' в начале
1 1 1 1; аааа; аааа	Стоимость 0; предписание М М М М	Полное совпадение
1 1 1 1; аааа; ааа	Стоимость 1; предписание D2 М М	D2 в начале
1 1 1 1; аааа; аа	Стоимость 2; предписание D2 D2	Двойное D2
1 1 1 1; аааа; а	Стоимость 3; предписание D2 D2 D	D2 дважды, затем обычное удаление
1 1 1 1; аааа; (пустая строка)	Стоимость 4; предписание D D D D	D2 в начале невозможна, четыре обычных удаления

Вывод

В ходе лабораторной работы реализован алгоритм вычисления расстояния Левенштейна с настраиваемыми стоимостями операций вставки, удаления и замены. В соответствии с вариантом 5б в алгоритм добавлена четвёртая операция — удаление двух последовательных символов, которая может применяться только тогда, когда первый из удаляемых символов совпадает с символом получаемой строки, стоящим непосредственно перед ними.

Программа выводит таблицы динамического программирования и операций, что позволяет проследить ход вычислений, а также восстанавливает и выводит последовательность операций, преобразующую исходную строку в целевую. Алгоритм имеет временную сложность $O(n \cdot m)$ и корректно обрабатывает граничные случаи: пустые строки, полное совпадение, полное несовпадение, а также ситуации, когда операция удаления двух символов допустима или, наоборот, запрещена условием варианта.